Mini-Projeto: Modelagem e solução de problemas com Aprendizado por Reforço

MountainCar: Q-learning, SARSA(λ) e Dyna-Q

Integrantes
Caetano gubert meneghetti
Fernando bosquetti tedesco
Gabriel de senne amorim
José antônio coutinho de moraes filho

CMP629 — Aprendizado por Reforço

1. Introdução e modelagem do ambiente

O objetivo é comparar aprendizagem e custo de três métodos tabulares sob o mesmo protocolo, identificando a variabilidade entre seeds e os limites da agregação de estados. A pergunta é como algoritmo e discretização se relacionam com o sucesso final e com o trabalho computacional medido. O mini-projeto segue o enunciado da CMP629 e os fundamentos de Sutton e Barto (2018, caps. 6, 8 e 12).

O MountainCar-v0 observa $s_t = (x_t, v_t)$, com posição em $[-1,2; 0,6]$ e velocidade em $[-0,07; 0,07]$. Ações: 0 (esquerda), 1 (neutra), 2 (direita). O carro deve atingir $x \geq 0,5$; o limite é 200 passos. O reset padrão sorteia a posição entre -0,6 e -0,4, com velocidade zero. A dinâmica usa força 0,001 e gravidade 0,0025; a velocidade é limitada e zerada na parede esquerda quando negativa.

$$v_{t+1} = \text{clip}(v_t + 0.001(a_t - 1) - 0.0025\cos(3x_t), -0.07, 0.07)$$

$$x_{t+1} = \text{clip}(x_t + v_{t+1}, -1.2, 0.6)$$

A recompensa é -1 em cada transição; o retorno episódico não descontado é $R = -T$.

O enunciado a descreve como esparsa: tecnicamente há custo denso e constante por passo, mas pouca informação direcional sobre como atingir a meta. O retorno descontado usado nas equações de valor não deve ser confundido com a soma não descontada registrada nos gráficos.

1.1 Duas representações tabulares

Tabela 1 — Definição das grades de discretização

Grade	Cortes internos de posição	Cortes internos de velocidade	Estados / valores Q
4×2	-0,6; -0,2; 0,2	0	8 / 24
12×4	-1,05; -0,90; -0,75; -0,60; -0,45; -0,30; -0,15; 0; 0,15; 0,30; 0,45	-0,02; 0; 0,02	48 / 144

Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

`np.digitize` atribui igualdade ao corte à célula superior. São grades não uniformes no domínio completo. A grade 4×2 distingue o sinal da velocidade; a 12×4 também distingue magnitudes próximas de zero. A tabela cresce seis vezes. Mais resolução pode reduzir agregação, mas aumenta o esforço de cobertura dos pares estado–ação.

1.2 Terminal e truncamento

No término verdadeiro o alvo não inclui bootstrap. No truncamento temporal, o código encerra a coleta do episódio, mas mantém o bootstrap. Isso aproxima o processo subjacente sem limite de tempo; não é solução exata de um MDP de horizonte finito com 200 passos, pois o tempo restante não integra o estado.

Exemplo conceitual de aliasing (não experimental): em 4×2, (-0,59; 0,001) e (-0,21; 0,06) pertencem à mesma célula de posição e velocidade, apesar de possuírem energias e futuros diferentes.

2. Algoritmos: definição e código executável

Tabela 2 — Classificação dos algoritmos

Método	Modelo	Política do alvo	Crédito temporal
Q-learning	model-free	off-policy: máximo sobre ações	TD de um passo
SARSA(λ)	model-free	on-policy: próxima ação ε-greedy	TD com traços acumulativos
Dyna-Q	model-based aprendido	off-policy: backups Q-learning	TD real e planejamento

Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

Todos são **value-based, tabulares, online, com ações discretas**, sem rede-alvo e sem replay buffer convencional. O modelo de contagens do Dyna-Q é memória para planejamento, não uma rede nem replay de minibatches. O treinamento é episódico com truncamento; a política de comportamento é estocástica ε-greedy, e a avaliação é greedy com desempate aleatório reprodutível. Não se afirma uma política determinística quando há empate.

2.1 Q-learning

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_b Q(s',b) - Q(s,a) \right]$$

O alvo greedy pode diferir da ação realmente escolhida: por isso off-policy.

2.2 SARSA(λ)

$$\delta_t = r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)$$
$$e_t(s_t, a_t) \leftarrow e_t(s_t, a_t) + 1$$
$$Q \leftarrow Q + \alpha \delta_t e_t, \qquad e \leftarrow \gamma \lambda e$$

Traços são zerados no início de cada episódio. O mesmo erro TD atualiza componentes visitados anteriormente; a próxima ação pertence à política de comportamento.

2.3 Dyna-Q

Cada interação produz um backup real, atualiza contagens do modelo e gera $N = 5$ backups simulados. O par (estado, ação) visitado é amostrado uniformemente; o desfecho (recompensa, próximo estado, terminal) é sorteado proporcionalmente à frequência observada. Guardar múltiplos desfechos evita sobrescrever tudo pela última transição, mas não torna a agregação Markoviana.

2.4 Implementação preservada

O notebook incorpora `src/mountaincar_rl/agents.py` do commit de referência. A única adaptação é tornar o import de Gymnasium opcional para a análise sem interação. As funções de treino e avaliação ficam definidas, **sem chamada automática**. O arquivo original sem adaptação está em `apoio/src/mountaincar_rl/agents.py`. Para uso futuro dessas funções, instalar `gymnasium==1.2.1`; novos treinamentos não fazem parte desta entrega.

As atualizações são baseadas em Sutton e Barto (2018). O Apêndice A reproduz trechos do código preservado, com identificação de arquivo e hash; a descrição não substitui os testes nem constitui nova execução experimental.

3. Desenho experimental e seleção de hiperparâmetros

Triagem: três algoritmos $\times$ seis configurações por método $\times$ duas grades $\times$ três seeds (100, 1100, 2100) = 108 execuções de 1.000 episódios. A seleção global usa a maior AUC média de treino nas duas grades. Não representa busca exaustiva nem ótimo universal. A avaliação não foi usada para selecionar.

Tabela 3 — Hiperparâmetros selecionados

Método	α **escolhido**	λ	ε **decay por episódio**	**N**
Q-learning	0,10	0	0,990	0
SARSA(λ)	0,02	0,90	0,995	0
Dyna-Q	0,05	0	0,995	5

Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

Q-learning testou $\alpha\in$\{0,05;0,1;0,2\} e decay$\in$\{0,99;0,995\}; SARSA testou $\alpha\in$\{0,02;0,05;0,1\}, $\lambda\in$\{0,7;0,9\}, decay=0,995; Dyna testou $\alpha\in$\{0,05;0,1\}, N$\in$\{1;5;10\}, decay=0,995. Valores selecionados favorecem a AUC global observada; α menor nos traços limita a magnitude da correção simultânea, uma justificativa mecanística, não uma ablação causal.

Final: 3×2×10 = 60 políticas independentes, 1.000 episódios por política. Seeds: 3100, 4100, 5100, 6100, 7100, 8100, 9100, 10100, 11100, 12100. Desconto γ=0,99; ε inicial 1, mínimo 0,02; decaimento multiplicativo por episódio. Resets de treino vêm de `SeedSequence([seed,1,episódio])`. A avaliação de cada política usa 100 resets comuns, 800000–800099, sem exploração. Os 6.000 episódios de avaliação não são 6.000 treinamentos independentes.

3.1 Estatística e definição dos gráficos

Para cada episódio, a unidade amostral é a seed de treinamento. A banda é o IC t pontual de 95%, com dez observações, e não desvio-padrão nem banda simultânea:

$$IC_{95\%} = \bar{y} \pm t_{0.975,n-1}\frac{s_y}{\sqrt{n}}, \qquad n = 10$$

Há curvas brutas por episódio e curvas móveis de 20 episódios para leitura da tendência. A suavização ocorre dentro de cada seed antes de calcular média/IC. No eixo de interações, interpola-se apenas no suporte comum observado **entre todas as 30 execuções da grade**; não há extrapolação.

Passos de um episódio malsucedido são registrados como 200, identificados como duração até sucesso **ou limite**. Eles não são chamados de sucesso. A duração estritamente condicional aos sucessos é apresentada separadamente, com denominador explícito; dados ausentes não viram zero.

A AUC do protocolo é a área normalizada da curva de treino $(R + 200)/200$ por episódio, não uma AUC por interações nem medida de avaliação independente. Inferência AUC usa Friedman por grade, Wilcoxon pareado por seed e Holm nas três comparações da mesma grade. Os resultados inferenciais existentes são preservados, sem nova seleção.

3.2 Avaliação independente

Tabela 4 — Resultados por algoritmo e discretização

Grade	Método	Média (%)	Mediana (%)	DP (pp)
4x2	Q-learning	38.20	0.00	49.41
4x2	SARSA(λ)	83.70	100.00	33.90
4x2	Dyna-Q	0.00	0.00	0.00
12x4	Q-learning	73.60	88.50	39.77
12x4	SARSA(λ)	100.00	100.00	0.00
12x4	Dyna-Q	31.80	1.50	46.70

Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

Tabela 5 — Resultados por algoritmo e discretização

Grade	Método	IC 95% inferior (%)	IC 95% superior (%)
4x2	Q-learning	2.85	73.55
4x2	SARSA(λ)	59.45	107.95
4x2	Dyna-Q	0.00	0.00
12x4	Q-learning	45.15	102.05
12x4	SARSA(λ)	100.00	100.00
12x4	Dyna-Q	-1.61	65.21

Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

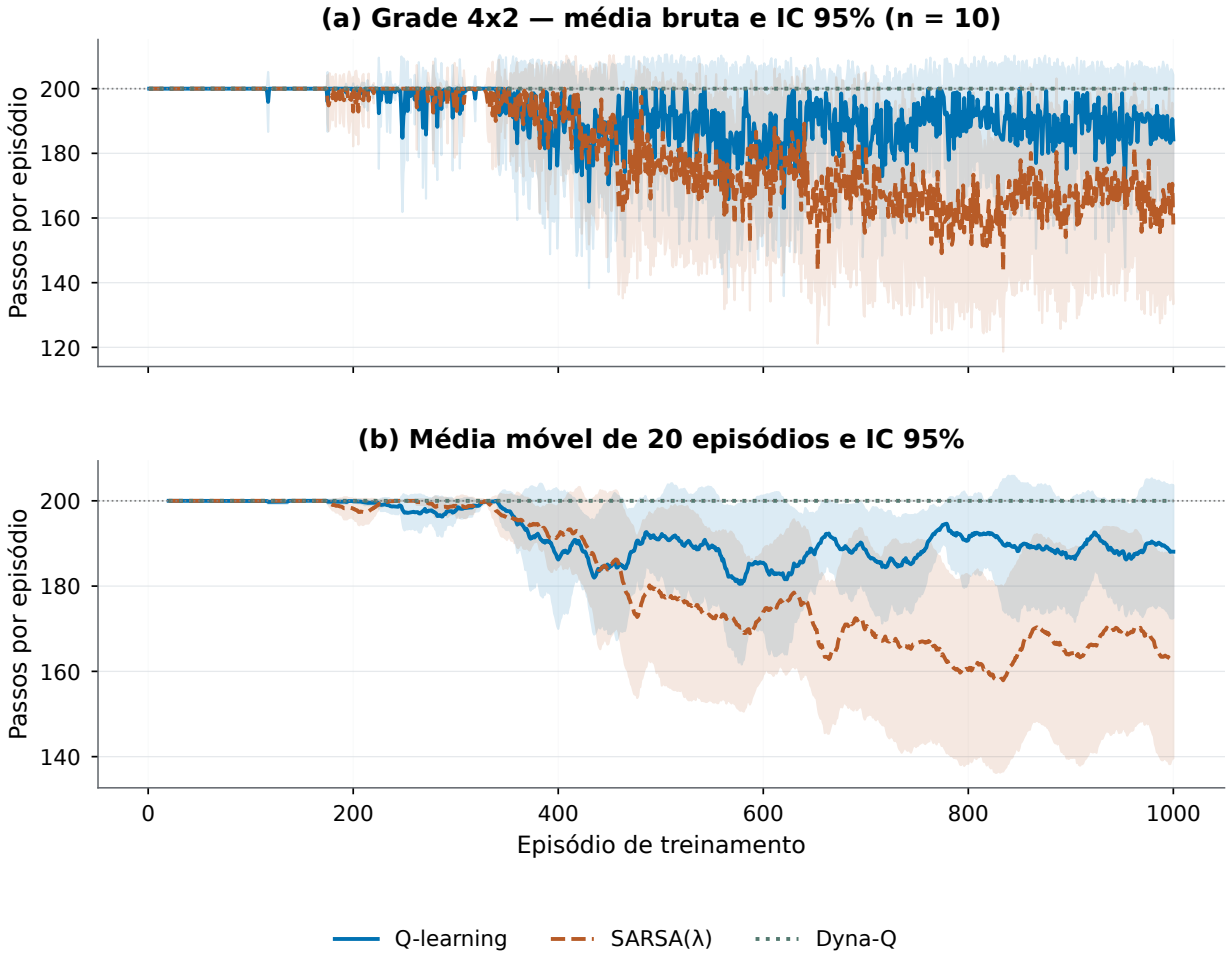
ICs t não truncados podem ultrapassar [0%,100%]. Um intervalo degenerado com DP zero não é garantia populacional.

4. Resultados: curvas exigidas pela rubrica

4.1 Duração por episódio: sucesso ou limite de 200 passos

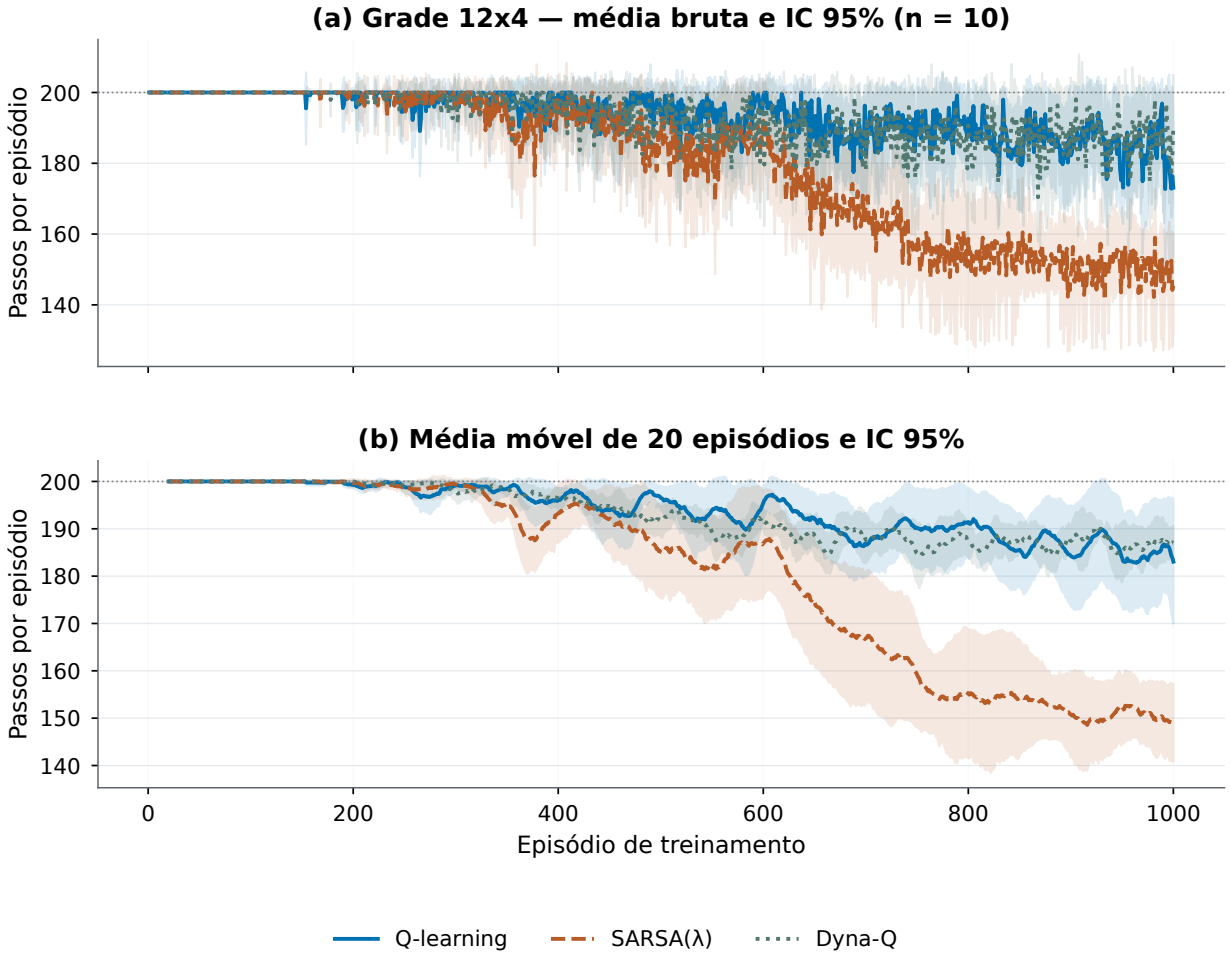
O painel superior mostra cada episódio, sem suavização, e o inferior a média móvel de 20 episódios. Todas as curvas usam dez seeds. ICs não são cortados artificialmente em 200; ultrapassar a faixa física evidencia a aproximação t. A linha horizontal marca o limite do ambiente.

Figura 1 — Duração por episódio e média móvel, grade 4x2



Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Média e IC t pontual de 95% entre dez seeds. Falhas consomem 200 passos; não são sucessos.

Figura 2 — Duração por episódio e média móvel, grade 12x4

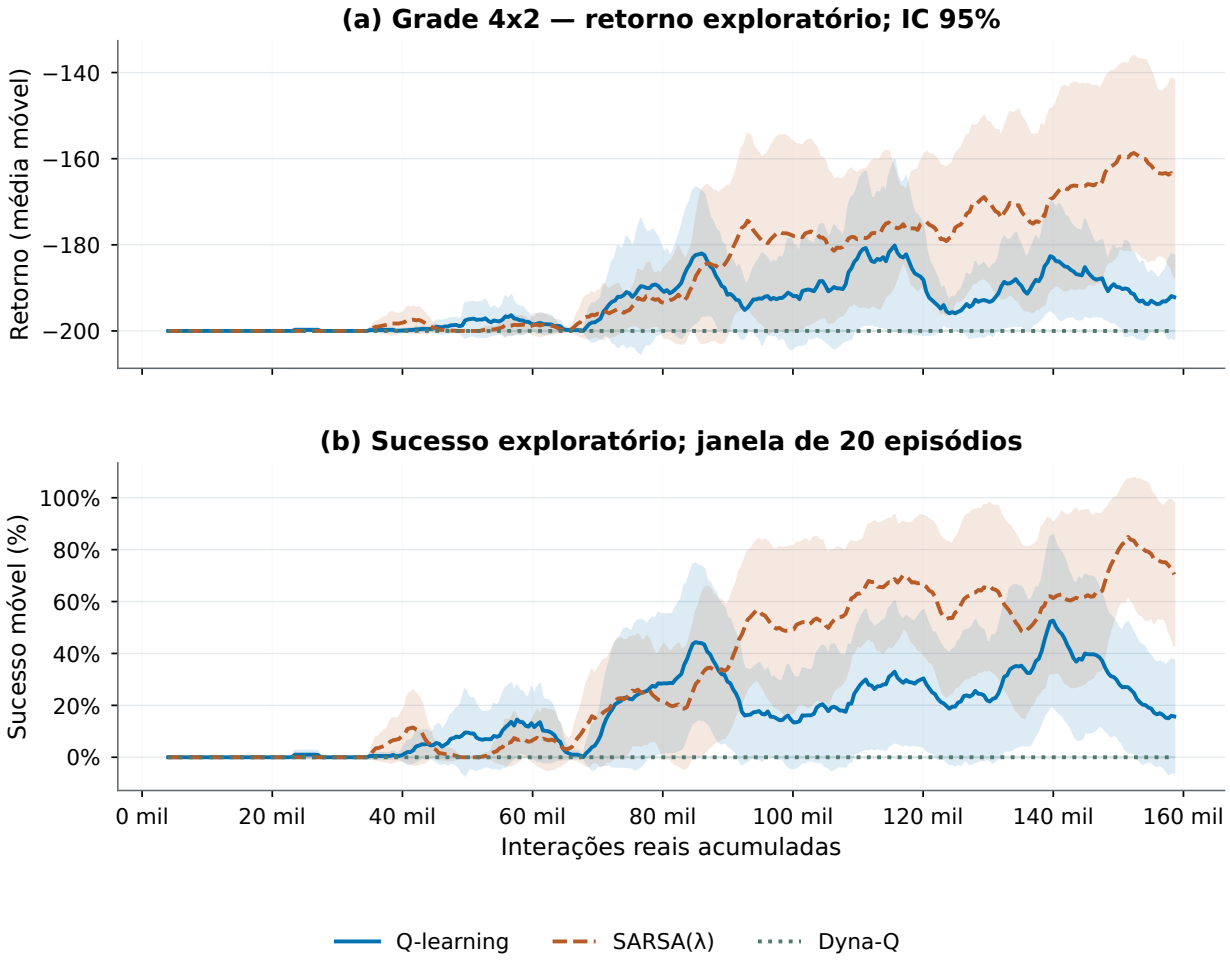


Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Média e IC t pontual de 95% entre dez seeds. Falhas consomem 200 passos; não são sucessos.

4.2 Retorno e sucesso por interações reais

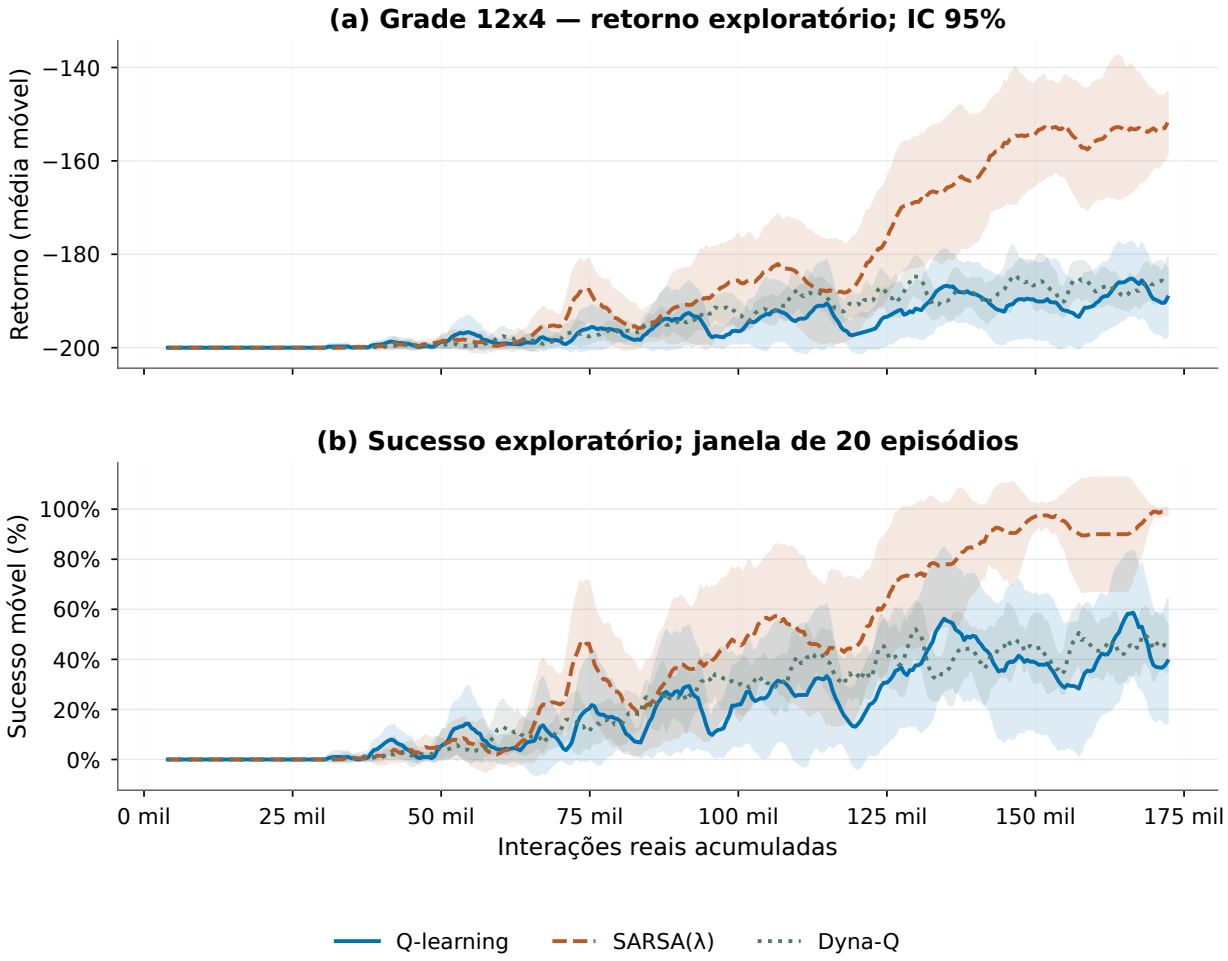
O desempenho do treino inclui exploração; ele não substitui avaliação greedy. O eixo horizontal conta somente chamadas reais a `env.step`, excluindo planejamento. O suporte comum evita comparar métodos em regiões sem observações de todas as seeds.

Figura 3 — Retorno e sucesso móvel por interações reais, grade 4x2; suporte comum às 30 execuções



Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Média e IC t 95%.

Figura 4 — Retorno e sucesso móvel por interações reais, grade 12x4; suporte comum às 30 execuções



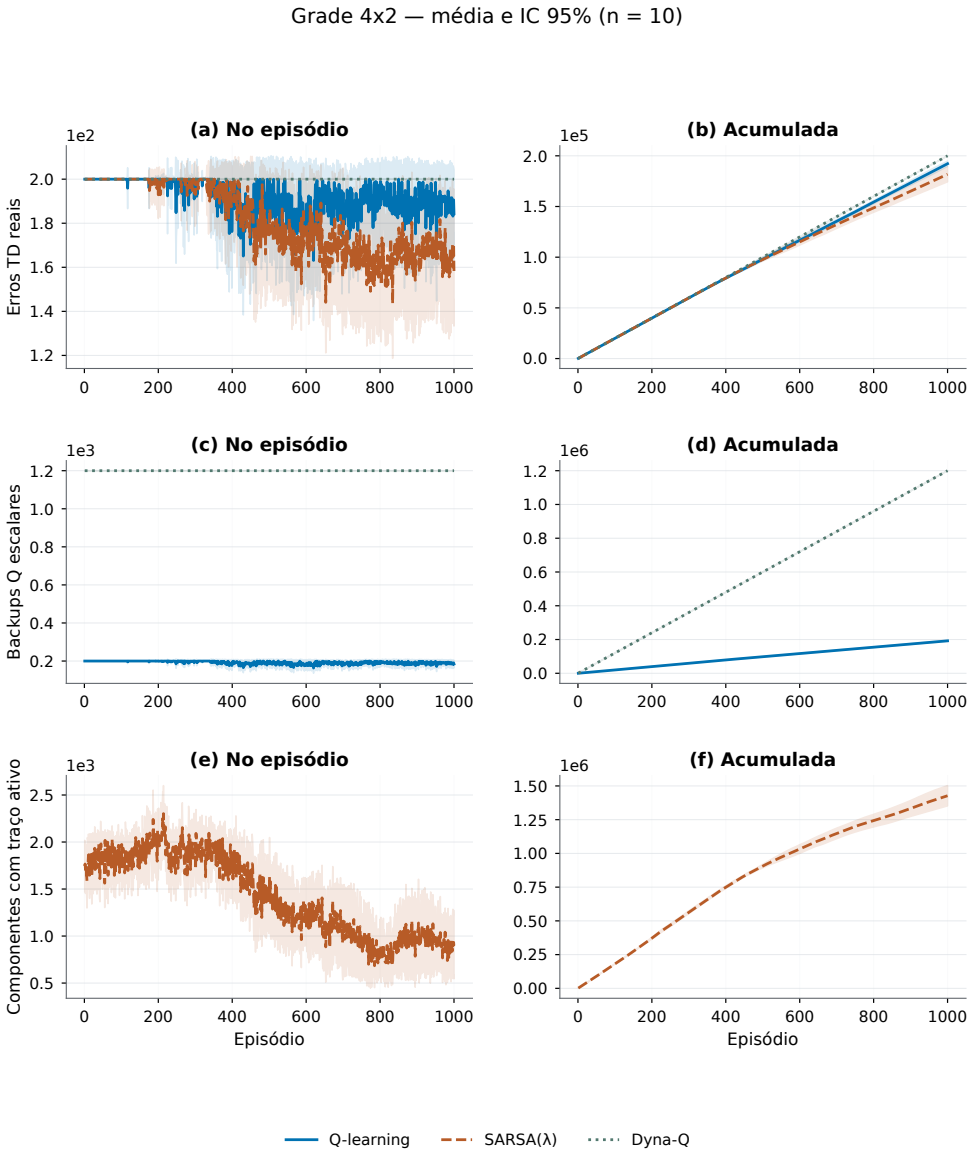
Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Média e IC t 95%.

4.3 Atualizações Q por episódio, com unidades explícitas

Os painéis separam (a) erros TD reais; (b) backups escalares de Q-learning e Dyna-Q, incluindo planejamento; (c) componentes com traço não nulo no SARSA(λ). Cada painel mostra contagem **no episódio** e **acumulada**, média e IC t 95%. Eixos verticais têm escalas próprias: não se comparam alturas entre unidades.

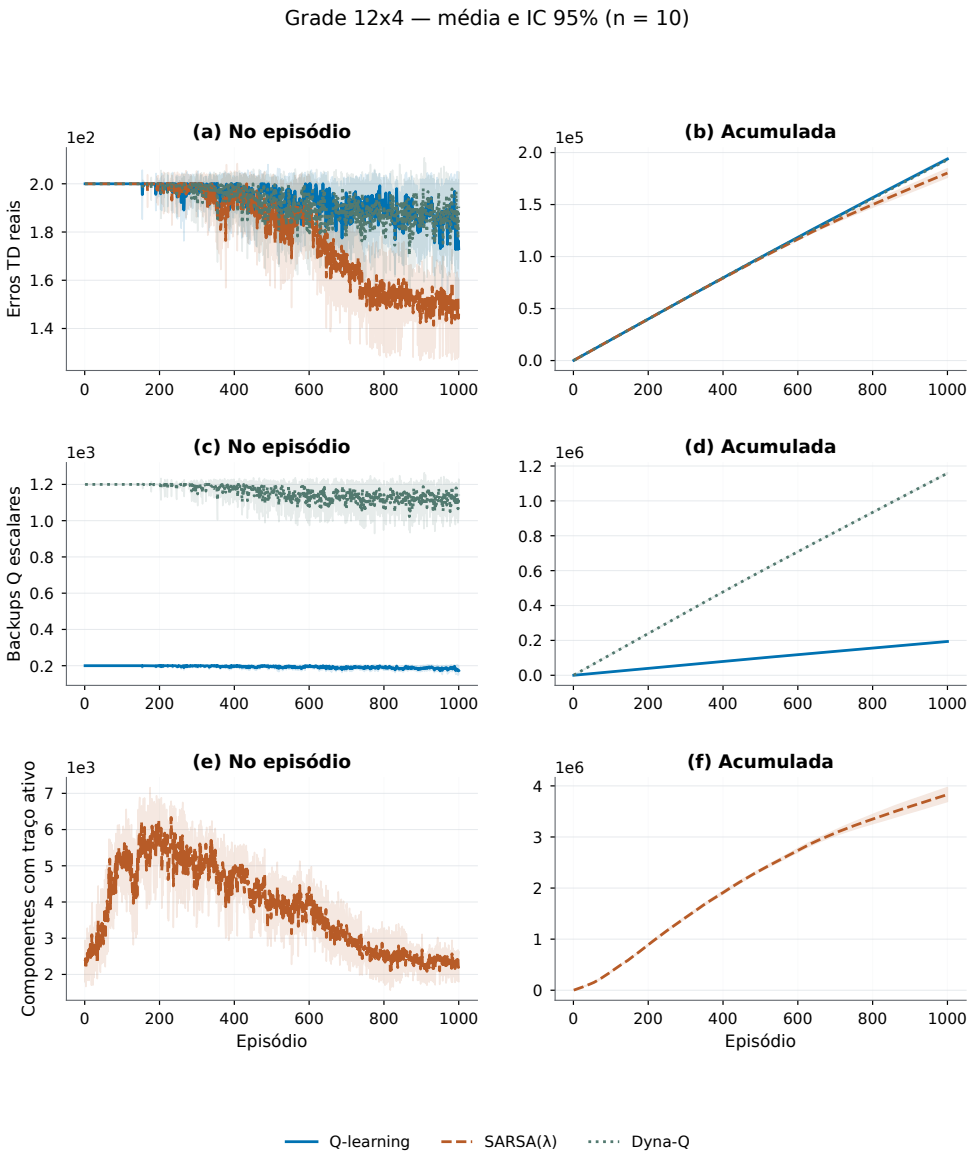
Em Dyna-Q, cada passo real gera $1 + N = 6$ backups escalares. Em SARSA, um erro TD modifica várias coordenadas. O código faz `q+=alpha*delta*traces` em um vetor denso; contar componentes não nulos **não mede** todas as operações aritméticas e de memória. Logo `total_q_updates` não é uma unidade universal de custo.

Figura 5 — Contagens por episódio e acumuladas, grade 4x2



Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Média e IC t 95%; unidades e escalas separadas.

Figura 6 — Contagens por episódio e acumuladas, grade 12x4



Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Média e IC t 95%; unidades e escalas separadas.

4.4 Duração estritamente condicional ao sucesso

Para esta métrica, cada política contribui com sua duração média apenas se teve sucesso. Políticas sem sucesso são ausentes. O quadro seguinte evita interpretar 200 passos de fracasso como “passos até o sucesso”. O IC continua entre políticas, com n variável.

Tabela 6 — Resultados por algoritmo e discretização

Grade	Método	n com sucesso / 10	Passos com falhas	Passos se sucesso
4x2	Q-learning	4	184.15	158.56
4x2	SARSA(λ)	9	161.77	153.19
4x2	Dyna-Q	0	200.00	—
12x4	Q-learning	8	167.06	156.21
12x4	SARSA(λ)	10	148.23	148.23
12x4	Dyna-Q	6	190.01	169.64

Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

Denominadores condicionais: 4, 9 e 0 políticas na grade 4$\times$2; 8, 10 e 6 na grade 12$\times$4, na ordem Q-learning, SARSA(λ), Dyna-Q. Ausência de sucesso não é duração zero. Os ICs completos estão no notebook e no CSV de duração.

5. Eficiência amostral e computacional

Eficiência amostral: SARSA(λ) apresentou a melhor combinação observada de AUC de treino, sucesso final e número total de interações nas duas grades. O orçamento fixou episódios, não um número idêntico de transições; atingir o objetivo cedo reduz o consumo de interações. Não há avaliações greedy intermediárias em orçamentos iguais de transições, portanto não se estima uma curva de generalização greedy ao longo do treino.

Eficiência computacional: Q-learning tem menos trabalho de atualização por passo; Dyna-Q paga pelo modelo e cinco backups adicionais. Nos tempos medidos, SARSA foi mais rápido na grade 4$\times$2, enquanto Q-learning e SARSA ficaram próximos em 12$\times$4. Não há vencedor universal: comparar sucesso e tempo conjuntamente. Contagens de traços não equivalem a backups escalares ou FLOPs.

A soma dos tempos individuais foi 3.492,42 s; com dois workers ela não é o tempo decorrido da campanha. Tempos dependem de hardware, carga e implementação. A latência registrada inclui consulta Q, argmax e desempate; **exclui discretização, transição do ambiente e renderização**. Ela não mede todo o ciclo de controle.

Tabela 7 — Resultados por algoritmo e discretização

Grade	Método	Interações reais	Backups modelo	Componentes traço
4x2	Q-learning	192199	0	0
4x2	SARSA(λ)	181792	0	1427374
4x2	Dyna-Q	200000	1000000	0
12x4	Q-learning	193815	0	0
12x4	SARSA(λ)	180296	0	3830204
12x4	Dyna-Q	193090	965448	0

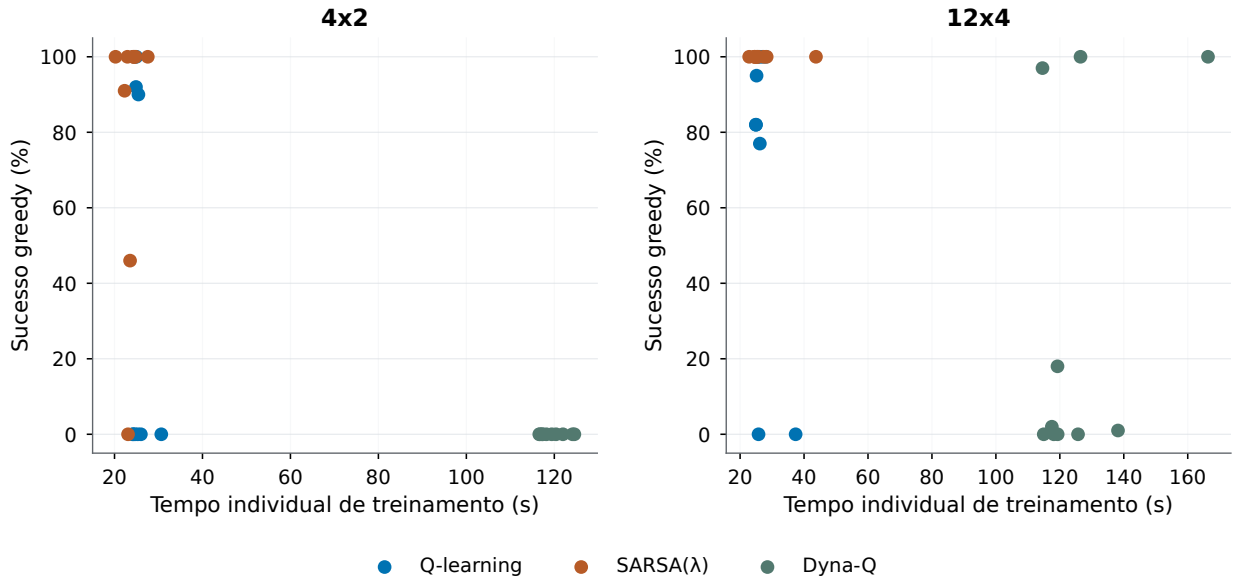
Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

Tabela 8 — Resultados por algoritmo e discretização

Grade	Método	Treino (s)	Seleção ação (μs)
4x2	Q-learning	25.472	49.212
4x2	SARSA(λ)	23.759	48.542
4x2	Dyna-Q	119.653	47.830
12x4	Q-learning	27.110	52.210
12x4	SARSA(λ)	27.206	56.417
12x4	Dyna-Q	126.042	51.659

Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

Figura 7 — Tempo individual de treino e qualidade greedy



Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Cada ponto representa uma política; não há média sobre episódios tratados como seeds.

Totais preservados: 11.411.906 interações reais, 19.654.480 backups simulados e 52.575.778 componentes com traços ativos. As categorias não devem ser somadas como operações equivalentes.

5.1 Distribuição entre seeds e percentis descritivos

A unidade de análise é a política treinada: **dez observações por algoritmo/grade**, cada uma avaliada em 100 resets comuns. Calculamos P10, P25, P50, P75 e P90 com interpolação linear (`numpy.quantile`, método `linear`, tipo 7). Para valores ordenados, o índice é $h = (n-1)p$; interpolam-se os vizinhos de h. O intervalo interquartil é $IQR = P75 - P25$. Percentis descrevem dispersão; **não são intervalos de confiança**. Com apenas dez seeds, P10/P90 são exploratórios e não estimam com precisão as caudas. Não calculamos P99 entre seeds.

A figura usa pontos para todas as seeds, caixa de P25 a P75 e traço central na mediana. Bigodes são mínimo/máximo, sem exclusão de pontos. O P95 de latência já gravado é um percentil das chamadas de seleção de ação de cada política; a mediana desses P95s **não equivale ao P95 global** nem mede todo o ciclo de controle. Tempos dependem da máquina e da carga. A análise é descritiva; não adiciona testes escolhidos após olhar os resultados.

Leitura dos percentis: em 4×2, Q-learning tem mediana de 0% e P75 de 91,5%, evidenciando forte heterogeneidade que a média de 38,2% resume mal. Em 12×4, Dyna-Q tem mediana de 1,5% apesar da média de 31,8%; poucas políticas de alto sucesso elevam a média. SARSA(λ) apresenta P25=P50=P75=100% na grade 12×4, um fato da

amostra observada, sem garantia populacional.

Tabela 9 — Percentis entre seeds: Sucesso greedy (%)

Grade	Método	P25	P50	P75	IQR
4x2	Q-learning	0.00	0.00	91.50	91.50
4x2	SARSA(λ)	93.25	100.00	100.00	6.75
4x2	Dyna-Q	0.00	0.00	0.00	0.00
12x4	Q-learning	78.25	88.50	100.00	21.75
12x4	SARSA(λ)	100.00	100.00	100.00	0.00
12x4	Dyna-Q	0.00	1.50	77.25	77.25

Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

Tabela 10 — Percentis entre seeds: Tempo de treino (s)

Grade	Método	P25	P50	P75	IQR
4x2	Q-learning	24.51	24.92	25.40	0.88
4x2	SARSA(λ)	22.98	23.87	24.56	1.59
4x2	Dyna-Q	117.27	118.80	121.56	4.30
12x4	Q-learning	25.25	25.93	26.81	1.56
12x4	SARSA(λ)	24.57	25.24	26.97	2.40
12x4	Dyna-Q	117.65	119.29	126.27	8.62

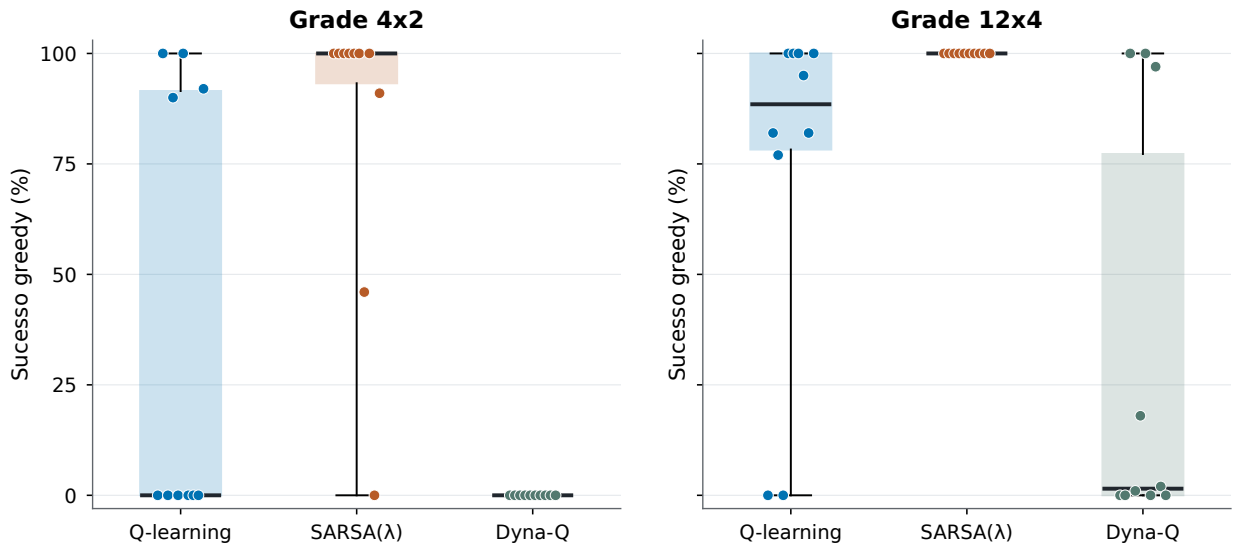
Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

Tabela 11 — Percentis entre seeds: P95 seleção de ação (μs)

Grade	Método	P25	P50	P75	IQR
4x2	Q-learning	79.15	80.65	83.80	4.65
4x2	SARSA(λ)	77.88	81.90	82.50	4.62
4x2	Dyna-Q	79.15	80.65	82.02	2.87
12x4	Q-learning	82.42	83.36	85.97	3.55
12x4	SARSA(λ)	80.85	84.45	97.05	16.20
12x4	Dyna-Q	77.28	80.70	84.60	7.32

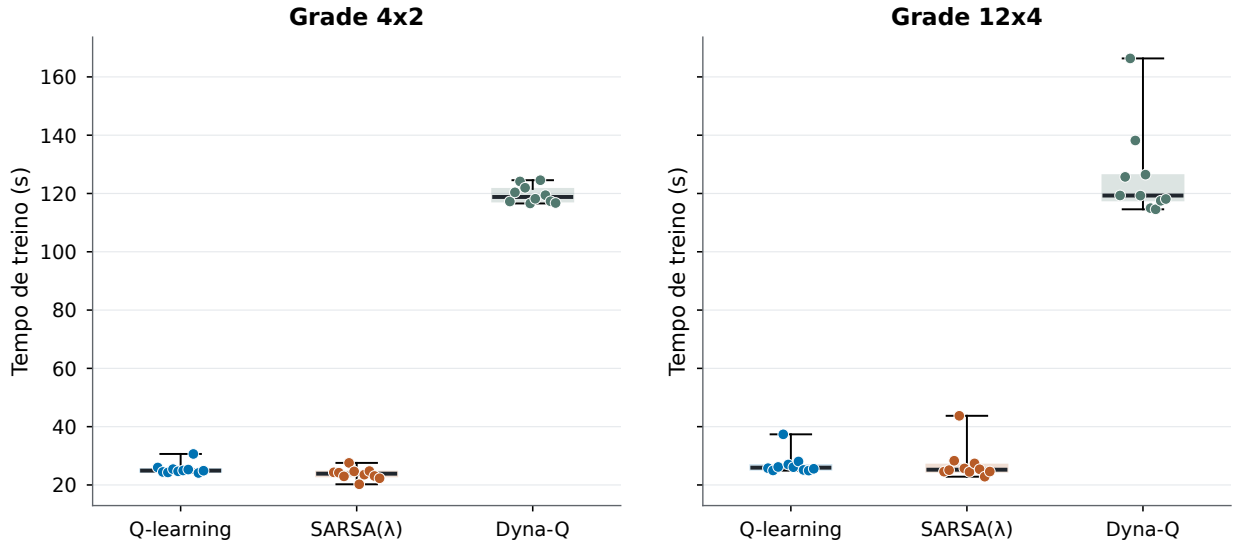
Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

Figura 8 — Distribuição do sucesso de avaliação entre políticas



Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Pontos: dez seeds; caixas: P25–P75; linha: mediana; bigodes: mínimo e máximo. Cada política foi avaliada em 100 episódios; não há exclusão de seeds.

Figura 9 — Distribuição do tempo de treinamento entre políticas



Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Pontos: dez seeds; caixas: P25–P75; linha: mediana; bigodes: mínimo e máximo. Tempos dependem de implementação e carga.

P10, P90, mínimos e máximos permanecem disponíveis no CSV `percentis_entre_seeds.csv` e no notebook. A tabela de P95 resume percentis de chamadas de cada política; não estima um percentil global de latência.

5.2 Inferência estatística predefinida

Os testes abaixo referem-se à **AUC de treinamento**, não ao sucesso de avaliação. O pareamento usa identificadores de seeds; as trajetórias não são idênticas entre métodos. A correção Holm controla três comparações por grade, não uma família global com ambas as grades. Intervalos bootstrap já publicados: 10.000 reamostragens pareadas, seed fixa. ICs degenerados de sucesso em 0% ou 100% descrevem a amostra e não garantem o mesmo resultado em futuras seeds.
Grade 12x4: Friedman = 15.200; p = 0.0005005.

Tabela 12 — Comparações pareadas da AUC

Comparação AUC	Diferença média	p Holm	IC bootstrap 95%
Q-learning - SARSA(λ)	-0.06760	0.005859	[-0.08457; -0.04887]
Q-learning - Dyna-Q	-0.00367	0.275391	[-0.00997; 0.00186]
SARSA(λ) - Dyna-Q	0.06392	0.005859	[0.04628; 0.08017]

Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

Grade 4x2: Friedman = 16.800; p = 0.0002249.

Tabela 13 — Comparações pareadas da AUC

Comparação AUC	Diferença média	p Holm	IC bootstrap 95%
Q-learning - SARSA(λ)	-0.05202	0.009766	[-0.08837; -0.02188]
Q-learning - Dyna-Q	0.03901	0.005859	[0.02858; 0.04944]
SARSA(λ) - Dyna-Q	0.09103	0.005859	[0.06333; 0.12382]

Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

6. State aliasing: Dyna-Q versus SARSA(λ)

Fato: a grade 4×2 agrega estados físicos distintos. Dyna-Q obteve 0% de sucesso de avaliação nessa grade, contra 31,8% em 12×4; SARSA obteve 83,7% e 100%. **Interpretação:** a representação mais fina distingue situações relevantes de posição e velocidade, mas a mudança também altera cobertura, tamanho da tabela e dinâmica da aprendizagem.

No Dyna-Q, o modelo aprende a distribuição de desfechos condicionada à célula agregada e à distribuição de visitas. A mesma ação pode gerar futuros distintos dentro de

uma célula. Quando a política muda, também pode mudar a mistura de estados físicos visitados. Repetir cinco vezes esse modelo pode propagar erro de agregação. Guardar contagens de múltiplos desfechos melhora a descrição empírica, mas não recupera a informação física perdida.

No SARSA(λ), os traços propagam crédito ao longo da trajetória realmente observada, sem simular uma sequência pelo modelo agregado. Isso pode ser vantajoso neste problema. Ainda assim, a própria função Q do SARSA sofre aliasing; traços não restauram a propriedade de Markov.

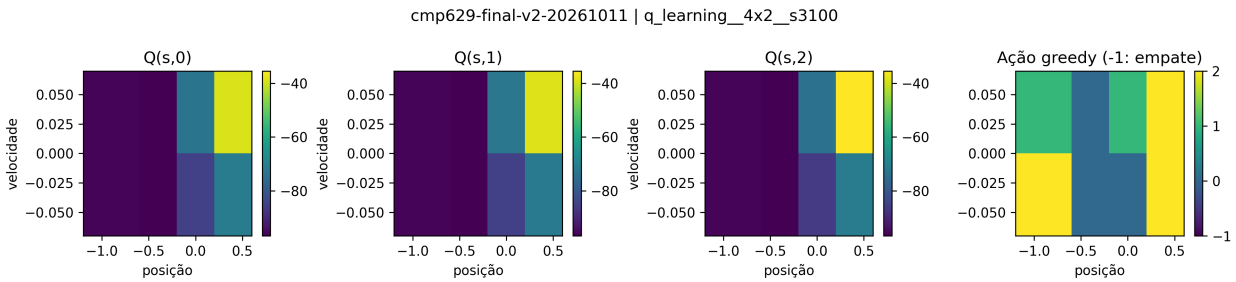
Hipótese não demonstrada: erro do modelo agregado explica o baixo desempenho do Dyna-Q. Para testar causalidade seriam necessárias ablações de resolução, N, representação/modelo e orçamento equivalente. Esta rodada não executa essas ablações e não atribui causalidade aos resultados.

7. Políticas, trajetórias e vídeos reais

As seis demonstrações usam a menor seed de treino (3100) e o reset 900000, escolhido previamente. Mostram êxitos e fracassos, sem selecionar a melhor execução. São rollouts de checkpoints reais preservados, não animações ilustrativas. Os vídeos já estão incorporados e não geram novas interações. A taxa de sucesso deve ser lida na avaliação de 100 episódios por política, nunca inferida de um único vídeo.

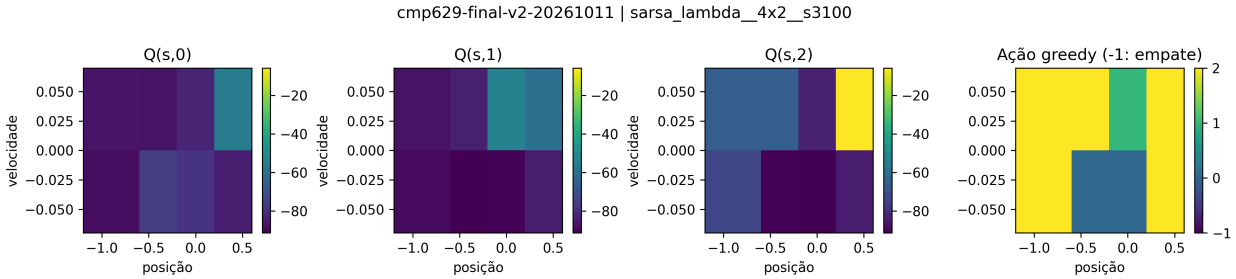
No PDF, os mapas abaixo documentam as políticas; os oito MP4s completos são fornecidos no pacote e as duas comparações são exibidas no notebook.

Figura 10 — Mapa de valores Q e política: q_learning, grade 4x2, seed 3100



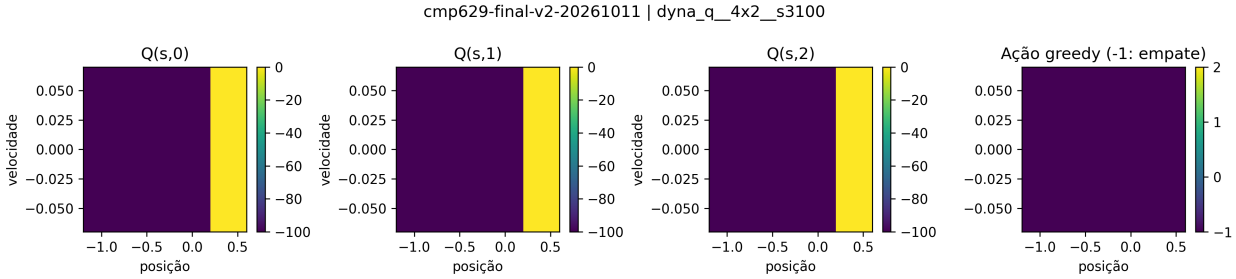
Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Células usam os limites físicos; empates estão identificados.

Figura 11 — Mapa de valores Q e política: sarsa_lambda, grade 4x2, seed 3100



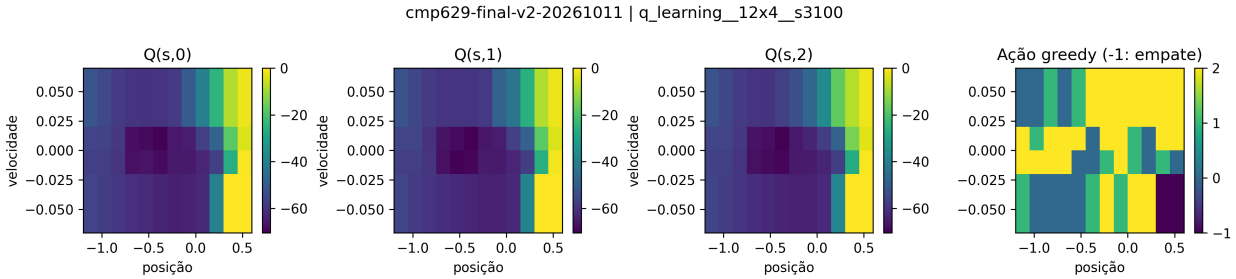
Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Células usam os limites físicos; empates estão identificados.

Figura 12 — Mapa de valores Q e política: dyna_q, grade 4x2, seed 3100



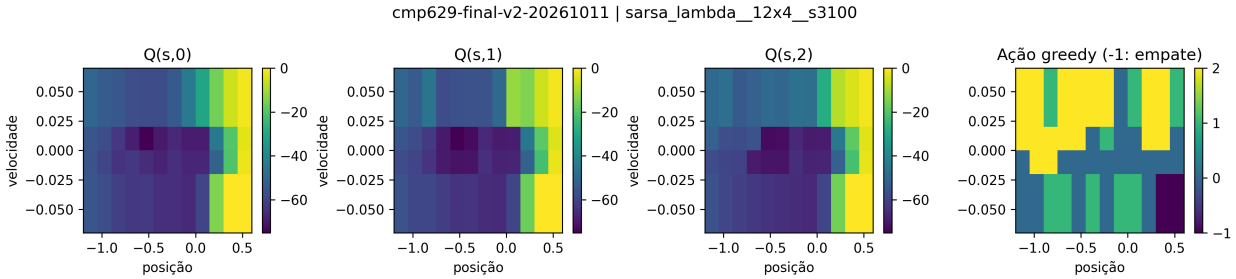
Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Células usam os limites físicos; empates estão identificados.

Figura 13 — Mapa de valores Q e política: q_learning, grade 12x4, seed 3100



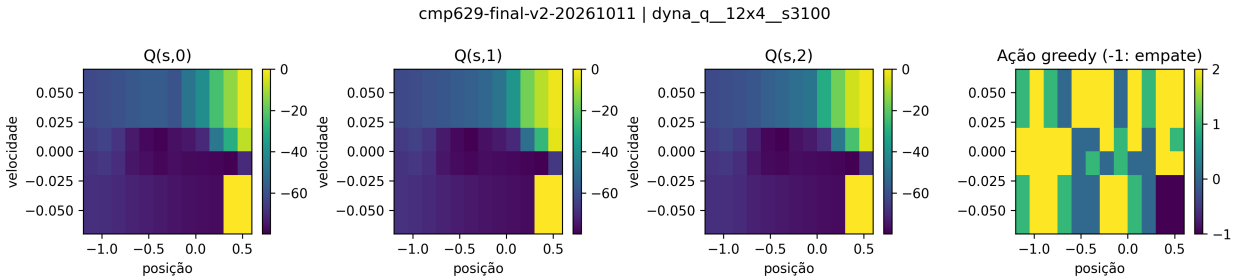
Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Células usam os limites físicos; empates estão identificados.

Figura 14 — Mapa de valores Q e política: sarsa_lambda, grade 12x4, seed 3100



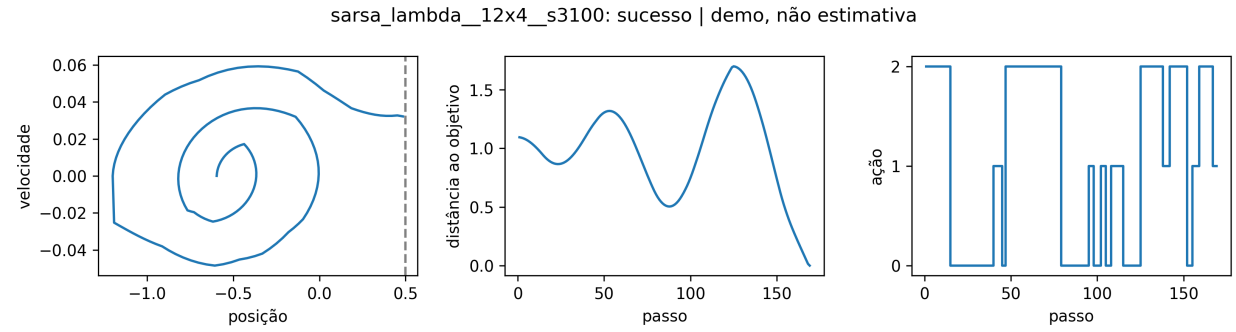
Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Células usam os limites físicos; empates estão identificados.

Figura 15 — Mapa de valores Q e política: dyna_q, grade 12x4, seed 3100



Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Células usam os limites físicos; empates estão identificados.

Figura 16 — Trajetória real de SARSA(λ), grade 12$\times$4, seed 3100, reset 900000



Fonte: Elaboração própria a partir dos históricos e checkpoints finais preservados (2026).
Nota: Um exemplo não estima desempenho médio.

8. Diário de bordo, tentativas inferiores e correções

Registro reconstruído a partir dos protocolos, históricos e auditorias preservados; não se inventam conversas ou cronologia pessoal.

Tabela 14 — Diário de bordo e decisões

Etapa / fonte	**Evidência ou dificuldade**	**Tratamento e limite**
Triagem v1	108 execuções, apenas três seeds por configuração	Usada para selecionar, não para inferência final
Dyna-Q 4×2	As 18 execuções exploratórias não tiveram sucesso de avaliação	Aumentar N dentro da grade testada não resolveu; causalidade do aliasing não demonstrada
Auditoria 11/10, `docs/AUDITORIA.md`	"stable_episode" representava primeiro atingimento; não garantia manutenção	Distinguir limiar inicial e janela final; não chamar de estabilidade sustentada
Auditoria de contadores	`total_q_updates` tinha unidades distintas	Separar TD real, planejamento e componentes de traços
Revisão da avaliação	Treino com ε não equivale à política greedy	Campanha final usa 100 resets comuns e dez seeds novas
Modelo Dyna	Risco de sobrescrever desfechos da célula pelo último observado	Código recebido já preserva distribuição empírica; não afirmar que o bug ocorreu no treino final
Entrega 13/10	Notebook anterior concentrava apresentação de PNGs e deixava explicações no relatório externo	Código incorporado, dados portáteis e gráficos gerados a partir dos históricos

Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

8.1 Configurações que tiveram desempenho inferior

A tabela abaixo mostra, por método, a melhor e a pior AUC média global entre as seis configurações da triagem. A ordenação é exploratória (seis execuções por configuração: duas grades $\times$ três seeds). Mudanças conjuntas de α, λ, decay e N não identificam efeitos individuais. Uma configuração "pior" nesta amostra não é universalmente ruim.

Mecanismos plausíveis: α elevado com traços acumulativos pode ampliar correções; exploração que cai depressa pode reduzir cobertura; N alto aumenta custo e pode reiterar um modelo agregado inadequado. Essas são explicações a investigar, não causas comprovadas pela triagem.

Tabela 15 — Configurações exploratórias

Método	α	λ	**Decay** ε	**N**	**AUC média**
Dyna-Q	0.05	0.0	0.995	5	0.017548
Dyna-Q	0.05	0.0	0.995	10	0.015485
Dyna-Q	0.05	0.0	0.995	1	0.011642
Dyna-Q	0.10	0.0	0.995	1	0.006449
Dyna-Q	0.10	0.0	0.995	5	0.005423
Dyna-Q	0.10	0.0	0.995	10	0.004308
Q-learning	0.10	0.0	0.990	0	0.046546
Q-learning	0.05	0.0	0.990	0	0.033045
Q-learning	0.20	0.0	0.990	0	0.028655
Q-learning	0.20	0.0	0.995	0	0.027991
Q-learning	0.05	0.0	0.995	0	0.025488
Q-learning	0.10	0.0	0.995	0	0.025098
SARSA(λ)	0.02	0.9	0.995	0	0.090396
SARSA(λ)	0.05	0.9	0.995	0	0.073456
SARSA(λ)	0.05	0.7	0.995	0	0.063098
SARSA(λ)	0.10	0.7	0.995	0	0.057698
SARSA(λ)	0.10	0.9	0.995	0	0.057333
SARSA(λ)	0.02	0.7	0.995	0	0.028105

Fonte: elaboração própria com base no protocolo e nos registros preservados do projeto (2026).

Fonte: 108 históricos de triagem, seis execuções por configuração. Ordenação por AUC média decrescente dentro de cada método. A comparação não controla isoladamente cada hiperparâmetro.

9. Conclusões, limitações e relação com MPC/RL

1. **Resultado sustentado:** SARSA(λ) teve maior sucesso médio de avaliação e AUC de treino nas duas grades deste protocolo. A variabilidade de Q-learning e Dyna-Q torna inadequado resumir o estudo por uma seed.
2. **Custos:** planejamento aumentou o trabalho e tempo do Dyna-Q; os traços do SARSA tiveram custo vetorizado. Atualizações de tipos diferentes não formam uma moeda computacional comum.
3. **Representação:** a resolução altera o problema aproximado; aliasing é explicação plausível, não isolada causalmente. Duas grades e um ambiente não sustentam generalização universal.
4. **Validade:** dez seeds, distribuição estreita de resets, hiperparâmetros globais, treino exploratório e avaliação apenas ao final; IC t aproximado; medições de tempo sob carga;

tempo não incluído no estado.
5. **MPC/RL:** a contribuição é metodológica: separar treinamento, planejamento e seleção de ação, registrar unidades e exigir qualidade comparável. Não houve MPC, QP ou KKT neste experimento. A dimensão física do estado permaneceu dois; aumentar a grade não é aumentar n_x. Nada aqui demonstra relações causais com N do horizonte MPC, dim(H), dim(KKT), ρ(A) ou norma de A. O N do Dyna-Q é número de backups, conceito distinto do horizonte MPC.

9.1 Cobertura da rubrica

Discretização (15%): seção 1 e verificação de cortes; agentes (25%): seção 2, código incorporado; rigor (10%): seção 3 e históricos preservados; gráficos (15%): seção 4; discussão (20%): seções 5–6 e 9; diário (15%): seção 8.

O enunciado valoriza compreensão e análise do estudante. Os integrantes devem revisar as interpretações e estar aptos a explicar as decisões metodológicas. Esta compilação declara o uso de assistência de IA na organização, implementação gráfica e revisão; não demonstra domínio individual por si só.

9.2 Reprodução e estado de validação

Históricos, código, vídeos, gráficos e outputs acompanham o pacote. A execução das células é registrada em VALIDACAO_EXECUCAO.json; não há treinamento. O Colab hospedado não foi testado; consulte `STATUS_ENTREGA.md`.

REFERÊNCIAS

SUTTON, Richard S.; BARTO, Andrew G. Reinforcement learning: an introduction. 2. ed. Cambridge, MA: MIT Press, 2018.

APÊNDICE A — CÓDIGO ESSENCIAL E RASTREABILIDADE

O código reproduzido pertence a `src/mountaincar_rl/agents.py`, do repositório de referência. SHA-256: 6e83091174ecb41592f05561ecdccab7aa1f2ec062 `7ba11029a5d8f618e598c8`. Os trechos são cópias literais, não novas implementações. A descontinuidade dos números de linha indica omissão de partes, não alteração do algoritmo.

Atualização escalar de Q

```
101 def scalar_q_update(
102     q: np.ndarray,
103     state: tuple[int, int],
104     action: int,
105     target: float,
106     alpha: float,
107 ) -> tuple[float, float]:
108     index = state + (action,)
109     old_value = float(q[index])
110     q[index] += alpha * (target - old_value)
111     return old_value, float(q[index])
```

SARSA com traços acumulativos

```
136 def sarsa_trace_update(q, traces, state, action, reward, next_state, next_action,
137                       terminated, alpha, gamma, lambda_):
138     """Accumulating traces; bootstrap on time limits, never on true terminal."""
139     bootstrap = 0.0 if terminated else gamma * q[next_state + (next_action,)]
140     delta = float(reward) + bootstrap - q[state + (action,)]
141     traces[state + (action,)] += 1.0
142     active = int(np.count_nonzero(traces))
143     q += alpha * delta * traces
144     traces *= gamma * lambda_
145     return float(delta), active
```

Backup de planejamento Dyna-Q

```
def planning_update(
    q: np.ndarray,
    model: EmpiricalTransitionModel,
    alpha: float,
    gamma: float,
    rng: np.random.Generator,
) -> dict[str, Any]:
    (state, action), (reward, next_state, terminated) = model.sample(rng)
    target = reward if terminated else reward + gamma * float(np.max(q[next_state]))
    old_value, new_value = scalar_q_update(q, state, action, target, alpha)
    return {
        "state": state,
        "action": action,
        "reward": reward,
        "next_state": next_state,
        "terminated": terminated,
        "target": target,
        "old_value": old_value,
        "new_value": new_value,
    }
```

Discretização do estado contínuo

```
class Discretizer:
    position_edges: tuple[float, ...]
    velocity_edges: tuple[float, ...]

    @classmethod
    def from_config(cls, config: dict[str, Any]) -> "Discretizer":
        spec = config["discretization"]
        return cls(tuple(spec["position_edges"]), tuple(spec["velocity_edges"]))

    @property
    def shape(self) -> tuple[int, int]:
        return len(self.position_edges) + 1, len(self.velocity_edges) + 1

    def transform(self, observation: np.ndarray | tuple[float, float]) -> tuple[int,
    ↪  int]:
        position, velocity = observation
        return (
            int(np.digitize(position, self.position_edges)),
            int(np.digitize(velocity, self.velocity_edges)),
        )
```

No Q-learning, o alvo usa o máximo em Q do próximo estado. Em SARSA, o próximo

valor é da ação escolhida pela política; os traços acumulam crédito e decaem por $\gamma\lambda$. O terminal real zera o bootstrap; o limite temporal não o zera. Em Dyna-Q, o modelo fornece transições simuladas e o alvo usa o máximo, preservando o custo extra de planejamento.

Verificação computacional dos percentis

```
# Uma observacao por politica treinada (10 seeds).
values = np.array([100*r['evaluation_metrics']['success_rate']
                   for r in group_runs(disc, alg)])
assert len(values) == 10 and np.isfinite(values).all()
p10, p25, p50, p75, p90 = np.quantile(
    values, [.10, .25, .50, .75, .90], method='linear')
iqr = p75 - p25
assert p10 <= p25 <= p50 <= p75 <= p90
```

APÊNDICE B — NORMALIZAÇÃO E LIMITES DA REVISÃO

A apresentação segue o modelo fornecido: papel Carta (612 × 792 pontos), margens de 2,54 cm, corpo sem serifa de 12 pontos, título de 24 pontos, seções de 18 pontos e subseções de 14 pontos; texto sem recuo inicial. A fonte Nimbus Sans substitui Arial, indisponível no ambiente. As páginas fluem sem capa, folha de rosto ou sumário adicional. Legendas, fontes e notas foram mantidas para identificação científica das figuras e tabelas. Os quatro integrantes foram transcritos conforme informados pelo grupo.

Os históricos registram o commit local de treino `f31e7b5744cd8f0610742c1ca8846ec74c597046` e `git_dirty=True`. Portanto, o identificador Git sozinho não prova a versão executada. O arquivo `relatorios/proveniencia_codigo.json` associa hashes SHA-256 dos módulos ao código publicado; os hashes e os bundles preservados sustentam a rastreabilidade, dentro dos limites do registro existente. As bibliotecas de treino registradas incluem Python 3.12.14, NumPy 2.3.3, Gymnasium 1.2.1 e SciPy 1.16.2, em Windows 10. O ambiente da análise atual está em `validacao_dados_notebook.json`; não deve ser confundido com o ambiente de treino.

Foram acrescentadas estatísticas descritivas e novas visualizações dos registros existentes, sem reestimar a campanha nem selecionar novos hiperparâmetros. O diário de bordo é uma reconstrução documental, não um registro contemporâneo inventado. As interpretações devem ser revisadas pelos integrantes; assistência de IA foi utilizada na edição, programação das análises e normalização. O teste no Google Colab hospedado permanece não executado.